Day(47)

* Anonymous or Lambda functions (most imp) *

→ Anonymous functions are those which does not contain any name explicitly.

→ The purpose of Anonymous Function is that "to perform instant operations".

→ Instant operations are those which are used at that point of time only and No longer interested to use in other part of the Applications/ project".

→ To define Anonymous function, we use a key word called lambda and Hence Anonymous functions are also called Lambda Functions.

→ Anonymous functions contains single executable statement (but not multiple statements)

→ Anonymous Functions automatically/implicitly returns the value (No need to use return statement).

syntax for Defining Anonymous functions:

```
varname = lambda params-list : statement
```

=> varname represents an object of type <class,'function'>. since varname acts as an object of function class and it can be used as function call.

> lambda is a keyword used for defining Anonymous Function.

params-list represents list of variable names used for storing the inputs coming from function call.

=> statement represents single executable statement and it gets executed and whose result returned automatically by the pvm (No need to use return statements).

Q. Defining a function for adding of two numbers.

By using Normal functions

```
def sumop(a,b):
    c = a+b
    return c
#main program
res = sumop(10,20)
print("sum=", res)
```

By using Anonymous Functions

```
addop = lambda a,b : a+b
#main program
res = addop(100,200)
print("sum=", res)
```

program for addition of two numbers by using normal and Anonymous functions.

Ex:

```
def sumop(a,b): #normal function
    c = a+b
    return c
add = lambda K,V : K+V # Anonymous functions

#main program
print("By using normal function:")
print("Types of sumop=", type(sumop)) # <class 'function'>
print("Enter Two values")
res = sumop(float(input()), float(input()))
print("sum=", res)
print("-  -  -  -")
print("By using AnonymousFunction:")
print("Types of addop=", type(addop)) #<class 'function'>
print("Enter two values")
res1 = addop(float(input()), float(input()))
print("sum=", res1)
```

Ex(1)

```
big=lambda a,b: a if a>b else b if b>a else "Both values equal"

#main program
print("Enter two values:")
a,b=float(input()),float(input())
bv=big(a,b)
print("big({},{})={}".format(a,b,bv))
```

Ex(2).

```
vowel=lambda word: "vowel word" if 'a' in word.lower()
                   or 'e' in word.lower() or 'i' in word.lower()
                   or 'o' in word.lower() or 'u' in word.lower()
                   else "NOT vowel word".

#main program
word=input("Enter word:")
res=vowel(word)
print("{} is {}".format(word,res))
```

* Comprehension:

→ Python is famous for allowing you to write code that's elegant, easy to write, and almost as easy to read as plain English. One of the language's most distinctive features is the list comprehension, which you can use to create powerful functionality within a single line of code rather than writing legacy lines of code.

→ The purpose of list comprehension is that to read the values dynamically from keyboard separated by a delimeter (space, comma, colon... etc.)

→ List comprehension is the most effective way for reading the data for list instead traditional reading the data.

syntax: listobj=[expression for varname in iterable_object if list cond]

=> Here expression represents either type casting or mathematical expression

e.g) print("Enter list of values separated by space:")
lst = [float(val) for val in input().split()]
print("Content of lst", lst)

e.g) lst = [4,3,7,-2,6,3]
newlst = [val*2 for val in lst]
print("new list =", newlst)

program for reading the values from the keyboard.

```
n=int(input("Enter How many values u want :"))
if(n<=0):
    print("{} is invalid input".format(n))
else:
    lst=[]
    for i in range(1,n+1):
        value=float(input("Enter {} value:".format(i)))
        lst.append(value)
    print("Content of list=",lst)
```

list

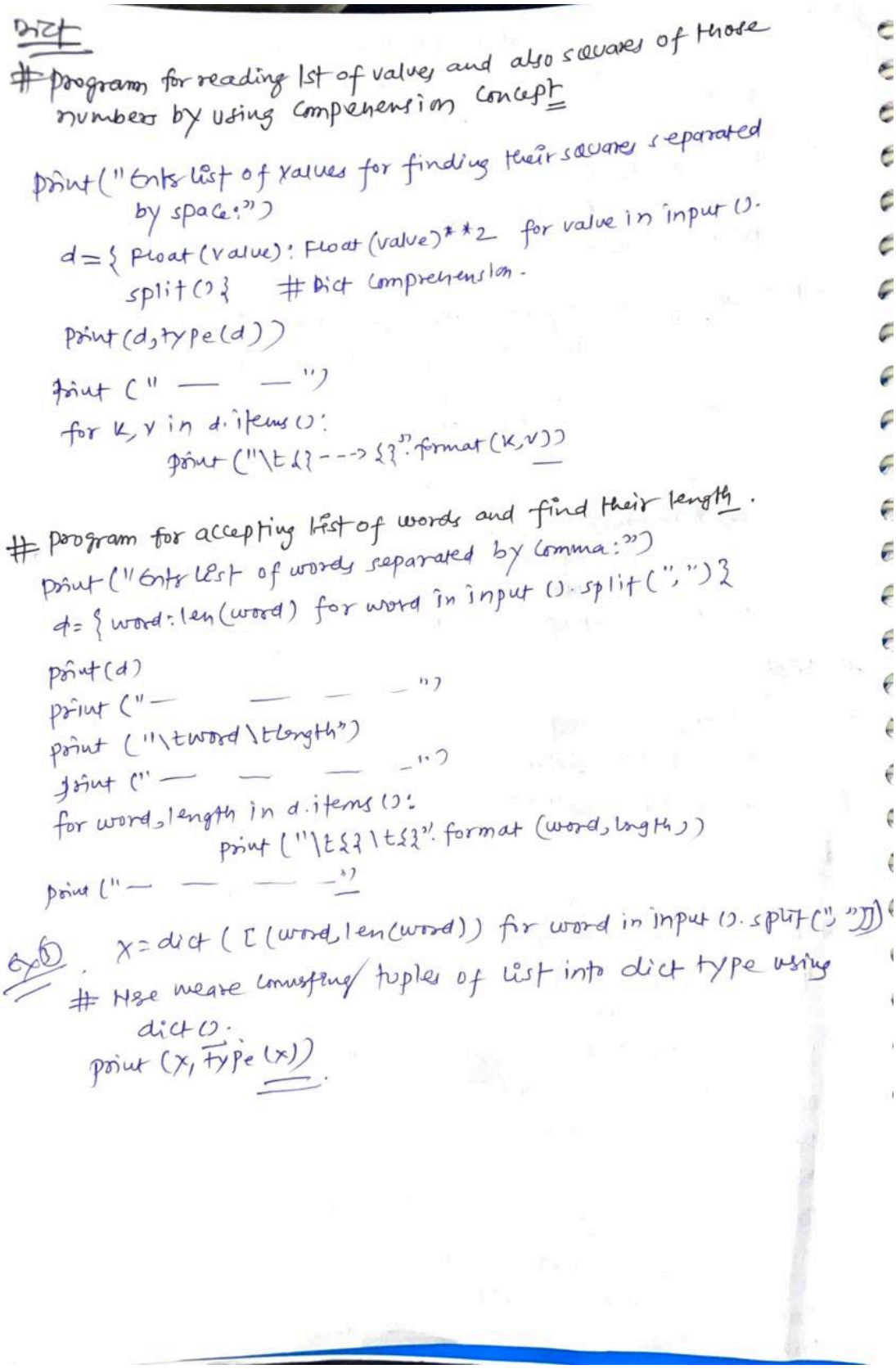
ex(1). print("Enter list of values separated by space:")
lst = [float(value) for value in input().split()] # list comprehension
print(lst, type(lst))

split(",")

ex(2).

ex(3). # program for accepting only +ve values from keyboard

```
print("Enter list of Numerical values separated by space for getting
      +ve vals:")
pslist=[float(value) for value in input().split() if float(value)>0]
print(pslist,type(pslist))
print("Number of +ve values=",len(pslist))
print("_____")
print("Enter list of numerical values separated by space for getting
      -ve vals:")
nglist=[float(value) for value in input().split() if float(value)<0]
print(nglist,type(nglist))
print("Number of -ve values=",len(nglist))
```



anonymous or lambda functions:-

- syntax:- varname=lambda params-list:statement

In [3]:
```
#program for addition of two numbers by using normal and anonymous functions
def sumop(a,b):          #normal function
    c=a+b
add=lambda k,v:k+v       #anonymous functions
```

```
#main program
print("by using normal function:")
print("Types of sumop=",type(sumop))      #<class,'function'>
print("enter two values")
res=sumop(float(input()),float(input()))
print("sum",res)
print("==========================================")
print("by using anonymousfunction:")
print("types of addop=",type(addop))
print("enter two values")
res1=addop(float(input()),float(input()))
print("sum =",res1)
```

```
by using normal function:
Types of sumop= <class 'function'>
enter two values
sum None
==========================================
by using anonymousfunction:
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[3], line 14
     12 print("==========================================")
     13 print("by using anonymousfunction:")
---> 14 print("types of addop=",type(addop))
     15 print("enter two values")
     16 res1=addop(float(input()),float(input()))

NameError: name 'addop' is not defined
```

In [4]:
```
#Program for Defining a Function for cal addition of Two Numbers
#AnonymousFunEx1.py
def   sumop(a,b): # Normal Function Definitiion
     c=a+b
     return c

addop=lambda a,b : a+b # Anonymous Function Definition

#Main program
print("Type of sumop=",type(sumop)) # Type of sumop= <class 'function'>
#Here sumop is an object of <class, function> and It can be used for as function
res=sumop(10,20) # Normal Function Call
print("Sum =",res)
print("======================================")
print("Type of addop=",type(addop)) # Type of addop= <class 'function'>
#Here addop is an object of <class, function> and It can be used for as function
res1=addop(100,200) # Anonymous Function Call
print("Sum=",res1)
```

```
Type of sumop= <class 'function'>
Sum = 30
======================================
Type of addop= <class 'function'>
Sum= 300
```

In [5]:
```
#Program for Defining a Function for cal addition of Two Numbers
#AnonymousFunEx2.py
addop=lambda a,b : a+b # Anonymous Function Definition
```

```
#Main program
print("Enter Two Values:")
a,b=float(input()),float(input())
c=addop(a,b)
print("sum({},{})={}".format(a,b,c))
print("---------------OR---------------------")
print("{} + {} = {}".format(a,b,addop(a,b)))
```

```
Enter Two Values:
sum(12.0,10.0)=22.0
---------------OR---------------------
12.0 + 10.0 = 22.0
```

In [6]:
```
#Program for Defining a Function for FINDING Biggest of Two Numbers
#AnonymousFunEx3.py

findbig=lambda a,b:a if a>b else b if b>a else "Both the Values are Equal" # Anc

#Main Program
print("Enter Two Values for Finding Big:")
a,b=float(input()),float(input())
bv=findbig(a,b)
print("Big({},{})={}".format(a,b,bv))
```

```
Enter Two Values for Finding Big:
Big(54.0,24.0)=54.0
```

In [7]:
```
#Program for Defining a Function for FINDING Biggest of Three Numbers
#AnonymousFunEx4.py

findbig=lambda a,b,c: a if (a>=b) and (a>c) else b if (b>a) and (b>=c) else c if
findsmall=lambda a,b,c: a if (a<=b) and (a<c) else b if (b<a) and (b<=c) else c
#Main Program
print("Enter Three Values for Finding Big:")
a,b,c=float(input()),float(input()),float(input())
bv=findbig(a,b,c)
sv=findsmall(a,b,c)
print("Big({},{},{})={}".format(a,b,c,bv))
print("Small({},{},{})={}".format(a,b,c,sv))
```

```
Enter Three Values for Finding Big:
Big(54.0,24.0,10.0)=54.0
Small(54.0,24.0,10.0)=10.0
```

In [9]:
```
## Program for Defining a Function for FINDING Max and Min from List of Numbers
#AnonymousFunEx5.py

maxv=lambda lst: max(lst)  # Anonymous Function
minv=lambda lstobj: min(lst) # Anonymous Function


#Main Program
print("Enter List of Values Separated by Space:")

lst=[float(val) for val in input().split()]
if(len(lst)==0):
        print("List is empty -can't find Max and Min")
elif(len(lst)==1):
        print("List Contains Single Value Max and Min={}".format(lst[0]))
elif(len(set(lst))==1):
        print("List Contains Same Values--all are equal")
```

```
else:
        mv1=maxv(lst)
        mv2=minv(lst)
        print("Max({})={}".format(lst,mv1))
        print("Min({})={}".format(lst,mv2))
```

```
Enter List of Values Separated by Space:
Max([54.0, 53.0, 55.0, 41.0, 78.0, 21.0, 42.0, 12.0])=78.0
Min([54.0, 53.0, 55.0, 41.0, 78.0, 21.0, 42.0, 12.0])=12.0
```

comprehension

In [2]:
```
#non-comphension
lst=[3,6,12,7,19,25]
sqlist=[]
for val in lst:
        sqlist.append(val**2)
else:
        print("Given List=",lst)
        print("Square  List=",sqlist)
```

```
Given List= [3, 6, 12, 7, 19, 25]
Square  List= [9, 36, 144, 49, 361, 625]
```

In [10]:
```
#ex1:
print("enter list of values separated by space:")
lst=[float(val)for val in input().split()]
print("content of lst",lst)
```

```
enter list of values separated by space:
content of lst [50.0, 41.0, 78.0, 98.0, 65.0, 47.0, 14.0, 25.0]
```

In [11]:
```
#ex2:
lst=[4,3,7,-2,6,3]
newlst=[val*2 for val in lst]
print("new list=",newlst)
```

```
new list= [8, 6, 14, -4, 12, 6]
```

In [12]:
```
#program for reading the values from keyboard:
n=int(input("enter how many values you want:"))
if(n<=0):
    print("{} is invalid inputt".format(n))
else:
    lst=[]
    for i in range(1,n+1):
        value=float(input("enter{} value:".format(i)))
        lst.append(value)
    print("content of list=",lst)
```

```
content of list= [87.0, 45.0, 41.0, 42.0, 43.0]
```

In [14]:
```
#list comprehension
#ex1:
print("enter list of values separated by space:")
lst=[float(value) for value in input().split()]
print(lst,type(lst))
```

```
enter list of values separated by space:
[41.0, 52.0, 47.0, 48.0, 49.0, 46.0, 43.0, 40.0] <class 'list'>
```

In [19]:
```
#Program for Reading List of Values by using Comprehension concept
#ListCompEx2.py
print("Enter List of  Values Separated by Comma:")
lst=[ float(value)  for value in input().split(",")]  # list Comprehension
print(lst,type(lst))
```

```
Enter List of  Values Separated by Comma:
[54.0, 87.0, 54.0, 95.0, 96.0, 93.0, 82.0] <class 'list'>
```

In [20]:
```
#program for accepting only +VE Values from Keyboard
#ListCompEx3.py
print("Enter List of Numerical Values separated by space for getting +Ve Vals:")
pslist=[ float(value)  for value in input().split()  if float(value)>0 ]
print(pslist,type(pslist))
print("Number of +Ve Values=",len(pslist))
print("-------------------------------------")
print("Enter List of Numerical Values separated by space for getting -Ve Vals:")
nglist=[ float(value)  for value in input().split()  if float(value)<0 ]
print(nglist,type(nglist))
print("Number of -Ve Values=",len(nglist))
```

```
Enter List of Numerical Values separated by space for getting +Ve Vals:
[54.0, 78.0, 45.0, 12.0, 46.0, 13.0, 49.0, 7.0, 2.0, 76.0, 73.0] <class 'list'>
Number of +Ve Values= 11
-------------------------------------
Enter List of Numerical Values separated by space for getting -Ve Vals:
[-78.0, -45.0, -1.0] <class 'list'>
Number of -Ve Values= 3
```

In [1]:
```
#Program for Reading lst of Values and also Squares of those numbers by using Co
#SetCompEx1.py
print("Enter list of  Values for finding their squares Separated by space:")
d={float(value):float(value)**2   for value in input().split() }  # Dict Compreh
print(d,type(d))
print("-----------------------------------------------")
for k,v in d.items():
    print("\t{}--->{}".format(k,v))
```

```
Enter list of  Values for finding their squares Separated by space:
{54.0: 2916.0, 47.0: 2209.0, 78.0: 6084.0, 41.0: 1681.0, 42.0: 1764.0, 43.0: 184
9.0} <class 'dict'>
-----------------------------------------------
        54.0--->2916.0
        47.0--->2209.0
        78.0--->6084.0
        41.0--->1681.0
        42.0--->1764.0
        43.0--->1849.0
```

In [3]:
```
#ReadValuesComprehension
#ex1:
print("Enter List of values separated by space:")
lst=[ int(val)   for val in input().split()]
print("Given List=",lst)
```

```
Enter List of values separated by space:
Given List= [54, 45, 24, 12, 14, 16, 15]
```

In [4]:
```
#ReadValuesComprehension
#Ex2.py
print("Enter List of Values separated by comma:")
```

```
lst=[ val  for val in input().split(",")]
print("Given List:{}".format(lst))
```

```
Enter List of Values separated by comma:
Given List:['54 41 42 43 46 49 14']
```

In [5]:
```
#dictcomphension
#ex1.py
lst=[3,6,12,7,19,25]
sqlist=dict([ (val,val**2)   for val in lst ])
print("Type of sqlist=",type(sqlist))
for n,s in sqlist.items():
        print("\t{}-->{}".format(n,s))
```

```
Type of sqlist= <class 'dict'>
        3-->9
        6-->36
        12-->144
        7-->49
        19-->361
        25-->625
```

In [6]:
```
#tuplecomphension
#ex1.py
lst=[3,6,12,7,19,25]
sqlist=( val**2   for val in lst )
print("Type of sqlist=",type(sqlist)) # <class 'generator'>
#Type casr generator object into tuple type
tpl=tuple(sqlist)
print("Type of tpl=",type(tpl))
print("Given List=",lst)
print("Square  List=",tpl)
```

```
Type of sqlist= <class 'generator'>
Type of tpl= <class 'tuple'>
Given List= [3, 6, 12, 7, 19, 25]
Square  List= (9, 36, 144, 49, 361, 625)
```

In [ ]:
```

```